Exception Handling in Taskflow

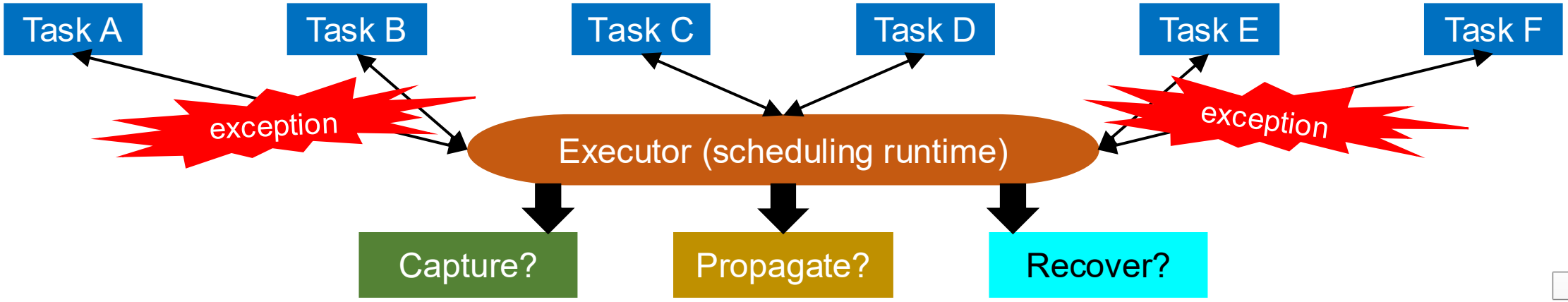
Dr. Tsung-Wei (TW) Huang
Department of Electrical and Computer Engineering
University of Wisconsin at Madison, Madison, WI
https://taskflow.github.io/

Takeaways

- Understand the core exception-handling logic of Taskflow
- Examine the algorithm flow of exception handling
- Walk through practical exception-handling examples
- Conclude

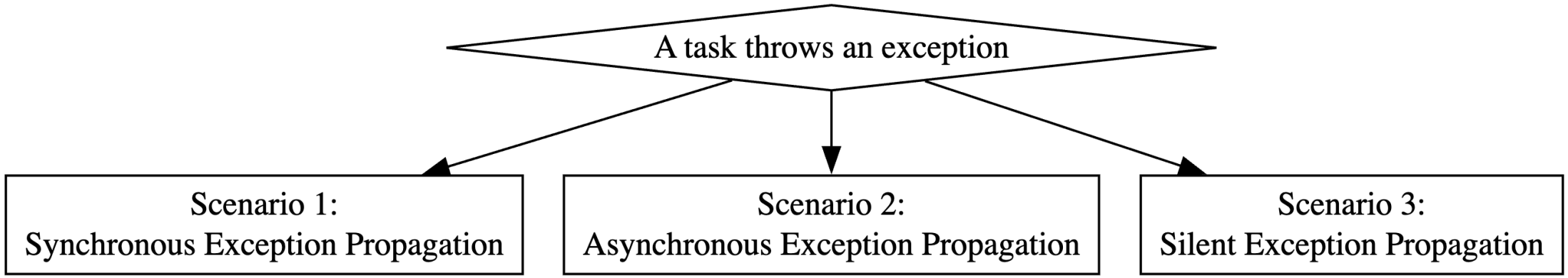
Exception Handling in Taskflow

- **A key feature that sets Taskflow apart from existing libraries that ignore it**
  - Ex: OpenMP has no standard mechanism to propagate exceptions across threads
    - Throwing an exception within an OpenMP parallel region is an undefined behavior
- **A major reason Taskflow is adopted in real-world production systems**
  - First requested by an industrial user in a semiconductor company
  - Later iterated, refined, and enhanced over time by various real-world use cases
- **A VERY challenging task in a multithreaded environment**
  - Multiple tasks can throw exceptions simultaneously in an unpredictable way



Three Exception Handling Scenarios in Taskflow[1]

- **A simple, robust design to handle exceptions in a task-parallel environment**
  - Scenario #1: Synchronous exception propagation
  - Scenario #2: Asynchronous exception propagation
  - Scenario #3: Silent exception propagation



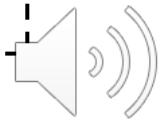
When multiple scenarios occur, Taskflow resolves them by applying scenario #1 first, then scenario #2, and finally scenario #3.

[1]: Exception handling in Taskflow: https://taskflow.github.io/taskflow/ExceptionHandling.html

Scenario #1: Synchronous Exception Propagation

- **Exceptions are propagated immediately to the caller thread**
  - The executor synchronously rethrows them without deferring the exception
  - Ex: Any exception occurs in `tf::Executor::corun` will be propagated to the caller

```
tf::Executor executor;
tf::Taskflow Taskflow;
taskflow.emplace([](){ throw std::runtime_error("exception"); });

executor.async([&](){
  // worker thread w1 issues a corun on the taskflow
  try {
    executor.corun(taskflow);
  }
  catch(const std::exception& e) {  // exception is rethrown to worker thread w1
    std::cerr << e.what();
  }
}).wait();
```



Scenario #2: Asynchronous Exception Handling

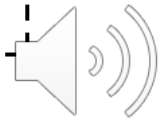
- **Exception is captured and propagated to the shared state**
  - This exception will later be rethrown when you access the result of the shared state (e.g., `std::future::get`)

```
tf::Executor executor;
tf::Taskflow Taskflow;

taskflow.emplace([](){ throw std::runtime_error("exception"); });
tf::Future<void> fu = executor.run(taskflow);

try {
  fu.get();
}
catch(const std::exception& e) {  // exception is propagated through the shared
  std::cerr << e.what();          // state associated with the future object
}
```

Scenario #3: Silent Exception Propagation

- **Exception is captured within its task if scenarios #1 and #2 do not apply**
  - Ex: The exception occurs in a `silent_async` task may be silently captured within the task

```
// A silent-async task with no parent context: exception silently suppressed within the task
executor.silent_async([&](){
  throw std::runtime_error("exception is silently caught by this task");
});

// A silent-async task with a parent context: exception propagated to the parent task group
executor.silent_async([&](){
  tf::TaskGroup tg = executor.task_group();
  tg.silent_async([](){
    throw std::runtime_error("exception is propagated to the parent task group");
  });
  try { tg.corun(); }
  catch(std::runtime_error& e) {
    std::cerr << e.what();
  }
});
```



Retrieve the Exception Pointer from a Task

- **When multiple tasks throw exceptions together, only one is propagated**
  - Other unpropagated exceptions will be silently stored within their respective tasks and can be later retrieved using `tf::Task::exception_ptr`

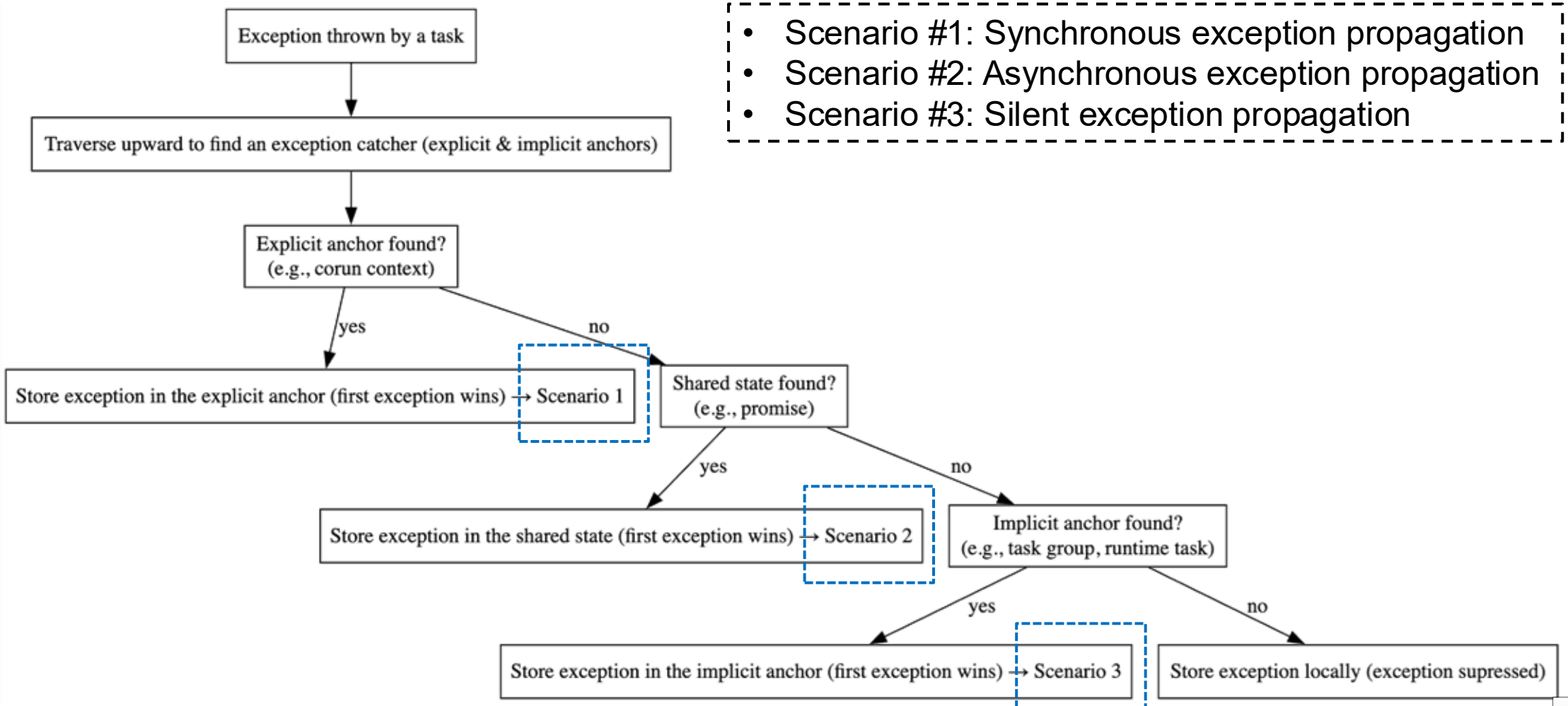
```
tf::Executor executor(2);
tf::Taskflow taskflow;
std::atomic<size_t> arrivals(0);
auto [B, C] = taskflow.emplace(
  [&]() { ++arrivals; while(arrivals != 2); throw std::runtime_error("oops"); },
  [&]() { ++arrivals; while(arrivals != 2); throw std::runtime_error("oops"); }
);
try {
  executor.run(taskflow).get();
}
catch (const std::runtime_error& e) {
  std::cerr << e.what();
}
// exactly one holds an exception as another was propagated to the try-catch block
assert((B.exception_ptr() != nullptr) != (C.exception_ptr() != nullptr));
```

Either the exception of `B` or `C` will be propagated to the shared state associated with the `future` object.

Takeaways

- Understand the core exception-handling logic of Taskflow
- Examine the algorithm flow of exception handling
- Walk through practical exception-handling examples
- Conclude

Algorithm Flow of Exception Handling



- Scenario #1: Synchronous exception propagation
- Scenario #2: Asynchronous exception propagation
- Scenario #3: Silent exception propagation

A Simple Way to Understand the Algorithm Flow

- **When a task throws an exception, ask the three questions below in order:**
  1. Is there any parent context waiting for the execution to complete?
     - Y: Write a try-catch block in that parent context to handle the exception
     - N: Go to the next question
  2. Is there any parent context associated with a shared state to which the exception can go?
     - Y: Write a try-catch block to access the result of the shared state
     - N: Go to the next question
  3. If neither applies, the exception is stored silently in the task
     - Retrieve the exception pointer from the task if needed
- **Exception handling in Taskflow has a non-deterministic behavior**
  - Taskflow can only propagate one exception although many tasks can throw simultaneously
  - Which exception is propagated is completely dynamic and depending on the runtime scheduler
  - You should never count on exception handling to perform control-flow logic

Takeaways

- Understand the core exception-handling logic of Taskflow
- Examine the algorithm flow of exception handling
- Walk through practical exception-handling examples
- Conclude

Catch an Exception from a Corun Loop

- **Scenario #1: Synchronous exception propagation**

```
tf::Executor executor;
tf::Taskflow taskflow1;
tf::Taskflow taskflow2;

taskflow1.emplace([](){ throw std::runtime_error("exception"); });

taskflow2.emplace([&](){
  try {
    executor.corun(taskflow1);
  } catch(const std::runtime_error& e) {
    std::cerr << e.what();
  }
});
executor.run(taskflow2).get();
```

The exception thrown by `taskflow1` will be caught by the corun loop, so it won't be propagated to the shared state associated with the future object of running `taskflow2`.

Catch an Exception from a Corun Loop (cont'd)

- **Scenario #1: Synchronous exception propagation**

```
tf::Executor executor;
tf::Taskflow taskflow1;
tf::Taskflow taskflow2;

taskflow1.emplace([](){ throw std::runtime_error("exception"); });

taskflow2.emplace([&](tf::Runtime& rt){
  try {
    rt.corun(taskflow1);
  } catch(const std::runtime_error& e) {
    std::cerr << e.what();
  }
});
executor.run(taskflow2).get();
```

We can observe the same behavior when using `tf::Runtime::corun`.

Catch an Exception from a Running Taskflow

- **Scenario #2: Asynchronous exception propagation**

When the task throws, the executor captures it and propagates it to the shared state associated with the returned `tf::Future` (a derived class of `std::future`). You can later catch the exception by calling the `get` method.

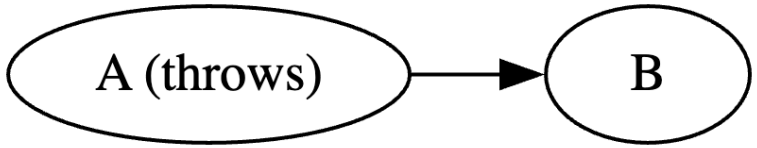
```
tf::Executor executor;
tf::Taskflow Taskflow;

taskflow.emplace([](){ throw std::runtime_error("exception"); });

// application thread t1 issues a corun on the taskflow
try {
  executor.run(taskflow).get();
}
catch(const std::exception& e) {  // exception is rethrown to thread t1
  std::cerr << e.what();
}
```

Catch an Exception from a Running Taskflow (cont'd)

- **Scenario #2: Asynchronous exception propagation**
  - When a task throws an exception, the executor will cancel the execution of its parent taskflow, i.e., all subsequent tasks that depend on the exception-throwing task will not be executed

```
int test = 0;
tf::Executor executor;
tf::Taskflow taskflow;
tf::Task A = taskflow.emplace([&](){ throw std::runtime_error("exception"); });
tf::Task B = taskflow.emplace([&](){ test = 1; });
A.precede(B);
try {
  executor.run(taskflow).get();
}
catch(const std::runtime_error& e) {
  std::cerr << e.what() << std::endl;
}
assert(test == 0);
```

When task A throws an exception, its parent taskflow will be cancelled by the executor, and thus task B won't be executed.



Catch an Exception from a Silent-async Task

- **Scenario #3: Silent exception propagation**

```
tf::Taskflow taskflow;
tf::Executor executor;

// exception will be silently ignored as there is no way to catch it
executor.silent_async([](){ throw std::runtime_error("exception"); });

// exception will be propagated to the parent taskflow
taskflow.emplace([&](tf::Runtime& rt){
  rt.silent_async([](){ throw std::runtime_error("exception"); });
});

try {
  taskflow.get();
}
catch(const std::runtime_error& e) {
  std::cerr << e.what();
}
```

Since `silent_async` does not return any future object, any exception thrown by a silent-async task will be (1) propagated to the its parent context if one exists or (2) stored silently within its task if that parent context does not exist.

Takeaways

- Understand the core exception-handling logic of Taskflow
- Examine the algorithm flow of exception handling
- Walk through practical exception-handling examples
- Conclude

Question?

Static Task Graph Programming (STGP)

```
// Live: https://godbolt.org/z/j8hx3xnnx

tf::Taskflow taskflow;
tf::Executor executor;
auto [A, B, C, D] = taskflow.emplace(
  [](){ std::cout << "TaskA\n"; }
  [](){ std::cout << "TaskB\n"; },
  [](){ std::cout << "TaskC\n"; },
  [](){ std::cout << "TaskD\n"; }
);

A.precede(B, C);
D.succeed(B, C);
executor.run(taskflow).wait();
```

Taskflow: https://taskflow.github.io

Dynamic Task Graph Programming (DTGP)

```
// Live: https://godbolt.org/z/T87PrTarx

tf::Executor executor;
auto A = executor.silent_dependent_async([]{
  std::cout << "TaskA\n";
});
auto B = executor.silent_dependent_async([]{
  std::cout << "TaskB\n";
}, A);
auto C = executor.silent_dependent_async([]{
  std::cout << "TaskC\n";
}, A);
auto D = executor.silent_dependent_async([]{
  std::cout << "TaskD\n";
}, B, C);
executor.wait_for_all();
```